C73-A3

changze wu,shuang wu

September 2024

1 Q1

1.1 a

The algorithm will terminate because each recursion the distant between l and r will become short, and finally each recursion will end with the base case and return to last stage.Let's proof by induction.

P(a): For any a = $r - l + 1$, $1 \leq a \leq len(A)$, the algorithm will sort the array A in slice between l and r.

Basis case: If $r = l$ than we done because there is only one element (line 2). If $r = l + 1$ we have only 2 element than we do line 4 to sort them, we also done. Thus, the base case hold.

I.H. Assume $3 \leq a < n$, P(a) hold. Than WTP: P(n) also holds.

We assume that a = n = 3k (we can separate them into $k_1, k_2, k3$ by their index Then length of each k_n is k, n is the sequence of each).

According to line 7, t = 2k. According to line 8 which is the first recursion of algorithm, the algorithm sort the 0 to 2k elements so that we can know every element in$k_1 \leq$ every element in k_2.(1) Then we do line 9, the second recursion, the algorithm makes every element in $k_2 \leq$ every elements in k_3.(2) WTP: after second recursion, every elements in k_3 is greater or equal than the k_1. For the second recursion doesn't affect the sorted k_1 in recursion 2. So the k_1 is same after recursion 1. We consider 2 case:

case 1: if all element in origin k_3 are less than k_2 after recursion 1. Then after recursion 2, every elements in k_3 is equal to k_2 after recursion 2. According to (1), we done.

case 2: if some element in origin k_3 are greater or equal to k_2 after recursion 1. Then every element in k_3 must be greater or equal than k_1 (1) (2). We also done

For case 1 and 2 and the recursion 3 didn't affect k_3, we can easily get final $k_1 \leq k_2 \leq k_3$ which is increasing order. **QED**

1.2 b

According to master theorem, when a approach to n:

$$T(n) = \begin{cases} 3 \cdot T(\frac{2n}{3}) + c * n^0, \ when \ n > 1 \\ c, \ when \ n = 1 \end{cases}$$

According to algorithm, we find that a = 3 (line 8-10), b = $\frac{3}{2}$ (line 7 - 10), d = 0 (line 8-10). According to mater theorem: $a < b^d$, $T(n) \in \Theta(n^{\log_b a}) = \Theta(n^{\log_{1.5} 3}) = \Theta(n^{2.709511})$

1.3 c

with comparing to bubble sort whose complexity is $\theta(n^2)$, this algo is worse than bubble sort.

2 Q2

2.1 a

Pre: For there are k-1 elements in couple and 1 singleton, we can get that n = 2(k-1) + 1 = 2k - 1, which $1 \leq k$ and $k \in \mathcal{R}$. n must be odd. In the first loop, A[l] is first element and A[r] is last elements.

Algorithm 1 SEARCH($\mathcal{A}, l, r$)

```
1: n' := r − l + 1
2: mid := ⌈n'/2⌉
3: if n' == 1 then
4:    return mid
5: end if
6: if A[mid] not equal to A[mid+1] and A[mid-1] then
7:    return mid
8: else if A[mid] == A[mid-1] then
9:    if mid is odd then
10:      return Search(A, l, mid-1)
11:   else
12:      return Search(A, mid+1, r)
13:   end if
14: else if A[mid] == A[mid+1] then
15:   if mid is odd then
16:      return Search(A, mid+1, r)
17:   else
18:      return Search(A, l, mid-1)
19:   end if
20: end if
```

we can find the answer through running A[1:n]

Running time: According to the algorithm, and suit for the master theorem, we can get a = 1(for we only consider one side) , b = 2 (find mid and separate into two) , d = 0 (all lines are if-else, which is constant), so

$$T(n) = \begin{cases} 1 \cdot T(\frac{n}{2}) + c * n^0, \ when\ n > 1 \\ c, \ when\ n = 1 \end{cases}$$

In this way, it is similar to binary search so we can get the complexity is $\Theta(logn)$ which is asymptotically faster.

Correctness: First of all we notice that the algorithm will terminate, since each recursion will make the r and l shorter and finally execute the base case (line 3). According to the Pre, we know that the $n = len(A)$ must be odd. Let prove this correctness by induction.

For the base case, if $n' = len(A) = 1$, it is obvious hold because line 4, the algorithm will return the index of first element, which is 0. I.H. Suppose for

arbitrary n', $1 \leq n' < n$, the algorithm will return the index of singleton. WTP: when n' = n, there is a singleton in A and the algorithm will return the position index of the singleton.

Then we consider three cases:

- CASE 1: A[mid] is the singleton, which is not equal to previous or next element, then we are done.

- CASE 2: A[mid] == A[mid - 1], if the mid is odd, then we should search the list before mid because each pair cost 2 place and the length of list before mid is odd and rest length is even, so the singleton must in the [l, mid-1]. Else if the mid is even, we search the rest list. In this case, we know that n' = n, and the [l, mid-1] or [mid+1, r] is equal to $[\frac{n}{2}]+1$ or $[\frac{n}{2}]-1$ which is clearly less than n and greater or equal to 1, according to IH, we hold.

- CASE 3: A[mid] == A[mid + 1]. This case is similar to CASE 2, which is symmetric. So we can easily hold.

According to the case 1, 2, 3, we can easily find that the algorithm will get the position of singleton in the A. QED.